
Current Topics in Digital Philology —

Assignment 5: Machine Translation for Historical Text

Daan Brugmans
Radboud University
daan.brugmans@ru.nl

1 Task 1

[See the task_1.py file]

2 Task 2

[Nothing to report]

3 Task 3

For task 3, I ran the historical Swedish test text through the trained SMT model and compared BLEU and exact match scores for the unaltered historical texts (“do nothing”) and the texts that were translated to modern Swedish by the NMT model, taking the modern Swedish test texts as the ground truth. The baseline BLEU score for the “do nothing” setup was 73.3, while the BLEU score for the translated text was 85.0, which means that the SMT model improved the BLEU score over the baseline. This can also be seen in the exact match metric: whereas the “do nothing” setup resulted in an exact match accuracy of 54.2%, the SMT-translated texts achieved an exact match accuracy of 76.4%, improving the accuracy by about 20 percent-points. The SMT-based translations thus show a clear improvement over the “do nothing” baseline.

4 Task 4

For my small experiment, I wanted to try to apply an already pretrained and finetuned mBART-50 model to the dataset. My main motivation for this is that I wanted to use an existing large neural machine translation model that already supported Swedish use its existing knowledge of Swedish to achieve high-performing results. I also had experience with using mBART-50 before, so I knew it would be feasible to achieve this.

I used an mBART-50 model through the HuggingFace platform, specifically facebook/mbart_large_50, and trained it on the exact same dataset and data splits as the SMT model. However, since this model is finetuned using entire sentences as training data instead of individual characters, I perform additional preprocessing that concatenates the word-split Swedish dataset provided for the assignment back into full sentences, including punctuation. Furthermore, since continuing the finetuning of the mBART-50 model in a regular way would take up a lot of time and energy, I instead finetune the model using LoRA, a Parameter-Efficient Finetuning (PEFT) method that significantly reduces the amount of resources used. Using HuggingFace’s LoRA support, I was able to finetune the mBART-50 model for 5 epochs within 5 minutes on an NVIDIA A6000 GPU.

However, my expectations did not meet my results, as the finetuned model achieved a very low BLEU score of 0.03, and an exact match accuracy of 0%. I expect that something must have gone wrong

during the training process. Although I tried to resolve the low scores by changing the training setup, this did not change the results. I propose a few possible explanations to what is happening:

- The model has a difficult time handling Swedish data as an input and an output at the same time.
- The limited size of the dataset prevented the model from training properly.
- The training parameters are somehow unfit for the training setup.

[illegible]

A Task 4 Code

NOTE: this code is also handed in as a .py file that may be easier to read.

```
from __future__ import annotations

import random
from abc import ABC
from pathlib import Path

import torch
import datasets
import transformers
import peft
import evaluate
from tqdm import tqdm

path_to_assignment_5 = Path(__file__).parents[0]
path_to_swedish_data = Path(path_to_assignment_5, "data", "swedish")

def set_universal_seed(seed: int) -> None:
    random.seed(seed)
    torch.manual_seed(seed)
    transformers.set_seed(seed)

    if torch.cuda.is_available():
        torch.backends.cudnn.benchmark = False
        torch.backends.cudnn.deterministic = False

def get_cuda_device() -> str:
    if torch.cuda.is_available():
        if torch.cuda.device_count() > 1:
            device = _get_cuda_device_with_most_free_memory()
        else:
            device = "cuda"
    else:
        device = "cpu"

    print(torch.device(device))

    if "cuda" in device:
        print(torch.cuda.get_device_name())
        print(
            f"{int(torch.cuda.get_device_properties(0).total_memory / 1000000000)} GB"
        )

    return device

def _get_cuda_device_with_most_free_memory() -> str:
    device = ""
    device_memory = 0

    for i in range(torch.cuda.device_count()):
        current_device = f"cuda:{i}"
        current_device_memory = torch.cuda.mem_get_info(current_device)[0]
```

```

        if current_device_memory >= device_memory:
            device = current_device
            device_memory = current_device_memory

    return device

class NMTModel(ABC):
    def __init__(self, use_custom: bool):
        super().__init__()

        self.use_custom = use_custom

        self.tokenizer = transformers.MBart50Tokenizer.from_pretrained(
            "facebook/mbart-large-50",
            use_fast=False,
        )

        base_model = transformers.MBartForConditionalGeneration.from_pretrained(
            "facebook/mbart-large-50"
        )

        if use_custom:
            self.mbart = peft.PeftModel.from_pretrained(
                base_model,
                Path(path_to_swedish_data, "models"),
            )
        else:
            self.mbart = base_model

        self.pad_id = self.tokenizer(
            "<pad>", add_special_tokens=False, return_tensors="pt", padding=True
        ).input_ids.item()

        self.bos_id = self.tokenizer(
            "<s>", add_special_tokens=False, return_tensors="pt", padding=True
        ).input_ids.item()

        self.eos_id = self.tokenizer(
            "</s>", add_special_tokens=False, return_tensors="pt", padding=True
        ).input_ids.item()

    def translate(
        self, source_sentence: str, source_lang: str, target_lang: str
    ) -> str:
        self.mbart.eval()

        tokenizer_input = source_sentence + f" </s> {source_lang}"
        model_input = self.tokenizer(
            tokenizer_input, add_special_tokens=False, return_tensors="pt", padding=True
        ).input_ids

        model_output = self.mbart.generate(
            inputs=model_input,
            use_cache=True,
            num_beams=4,
            max_length=60,
            min_length=1,
            early_stopping=True,

```

```

        pad_token_id=self.pad_id,
        bos_token_id=self.bos_id,
        eos_token_id=self.eos_id,
        decoder_start_token_id=self.tokenizer.lang_code_to_id[target_lang],
    )

    translated_sentence = self.tokenizer.decode(
        model_output[0],
        skip_special_tokens=True,
        clean_up_tokenization_spaces=False,
    )

    return translated_sentence

def translate_batch(
    self, dataset: datasets.Dataset, source_lang: str, target_lang: str
) -> tuple[list[str], list[str], list[str]]:
    sources = []
    targets = []
    predictions = []

    for row in tqdm(dataset):
        sources.append(row["hs"])
        targets.append(row["sv"])
        predictions.append(self.translate(row["hs"], source_lang, target_lang))

    return sources, targets, predictions

def get_bleu(
    self, dataset: datasets.Dataset, source_lang: str, target_lang: str
) -> tuple[float, dict, tuple[list[str], list[str], list[str]]]:
    sources, targets, predictions = self.translate_batch(
        dataset, source_lang, target_lang
    )

    preprocessed_targets = remove_removable_chars(targets)
    preprocessed_predictions = remove_removable_chars(predictions)

    preprocessed_targets = list(map(lambda x: x.lower(), preprocessed_targets))
    preprocessed_targets = list(map(lambda x: [x], preprocessed_targets))

    preprocessed_predictions = list(
        map(lambda x: x.lower(), preprocessed_predictions)
    )

    bleu = evaluate.load("bleu")
    bleu_results = bleu.compute(
        references=preprocessed_targets, predictions=preprocessed_predictions
    )

    return bleu_results["bleu"], bleu_results, (sources, targets, predictions)

def create_custom_dataset():
    for split in ["train", "test", "dev"]:
        dataset_dict = {}

        for version in ["hs", "sv"]:
            with open(

```

```

        Path(path_to_swedish_data , f"swedish.hs-sv.{ split }.{ version }"),
        "r",
        encoding="utf-8",
    ) as swedish_text_file:
        swedish_text = swedish_text_file.readlines()

        if split == "test":
            swedish_text = swedish_text[:2999]

    sentences = []
    current_sentence = []
    for line in swedish_text:
        if not line:
            current_sentence_str = " ".join(current_sentence)
            current_sentence_str = current_sentence_str.replace(" .", ".")
            current_sentence_str = current_sentence_str.strip()

            sentences.append(current_sentence)
            current_sentence = []

        if line == "\n":
            current_sentence_str = " ".join(current_sentence)
            current_sentence_str = current_sentence_str.replace(" .", ".")
            current_sentence_str = current_sentence_str.strip()

            sentences.append(current_sentence_str)
            current_sentence = []

        line = line.replace("\n", "")
        current_sentence.append(line)

    dataset_dict[f"{ version }"] = sentences

dataset_split = datasets.Dataset.from_dict(dataset_dict)
dataset_split.save_to_disk(str(Path(path_to_swedish_data , "nmt", f"{ split }"))

def train(
    hs_sv_model: NMTModel, train: datasets.Dataset, validation: datasets.Dataset
) -> None:
    def prepare_dataset_for_training(dataset: datasets.Dataset) -> datasets.Dataset:
        source_sentences = [row for row in dataset["hs"]]
        target_sentences = [row for row in dataset["sv"]]

        tokenized_sources = hs_sv_model.tokenizer(
            source_sentences ,
            max_length=128,
            truncation=True,
            padding="max_length",
            return_tensors="pt",
        )

        tokenized_targets = hs_sv_model.tokenizer(
            target_sentences ,
            max_length=128,
            truncation=True,
            padding="max_length",
            return_tensors="pt",
        ).input_ids

```

```

        tokenized_sources["labels"] = tokenized_targets

    return tokenized_sources

lora_config = peft.LoraConfig(
    r=8,
    lora_alpha=16, # Usually twice the rank 'r'
    lora_dropout=0.1,
    bias="none",
    task_type=peft.TaskType.SEQ_2_SEQ_LM,
    inference_mode=False,
    target_modules=["q_proj", "k_proj", "v_proj", "out_proj"],
)

training_args = transformers.Seq2SeqTrainingArguments(
    output_dir=None,
    eval_strategy="epoch",
    num_train_epochs=5,
    learning_rate=1e-3,
    weight_decay=1e-5,
    lr_scheduler_type="cosine",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    predict_with_generate=True,
    save_total_limit=1,
    fp16=True,
    seed=31313131,
)

lora_mbart = peft.get_peft_model(hs_sv_model.mbart, lora_config)
tokenized_train = train.map(prepare_dataset_for_training, batched=True)
tokenized_validation = validation.map(prepare_dataset_for_training, batched=True)

trainer = transformers.Seq2SeqTrainer(
    lora_mbart,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_validation,
    tokenizer=hs_sv_model.tokenizer,
)

lora_mbart.train()
trainer.train()
lora_mbart.save_pretrained(str(Path(path_to_assignment_5, "models")))

if __name__ == "__main__":
    set_universal_seed(31313131)
    device = get_cuda_device()

    nmt_model = NMTModel(False)
    print(nmt_model.mbart.__class__)

    create_custom_dataset()

    train_data = datasets.load_from_disk(
        str(Path(path_to_swedish_data, "nmt", "train"))
    )

```

```

test_data = datasets.load_from_disk(str(Path(path_to_swedish_data, "nmt", "test")))
dev_data = datasets.load_from_disk(str(Path(path_to_swedish_data, "nmt", "dev")))
train(nmt_model, train_data, dev_data)

custom_nmt_model = NMTModel(True)
print(custom_nmt_model.mbart.__class__)
bleu_score, bleu_details, bleu_data = custom_nmt_model.get_bleu(
    train_data, "sv_SE", "sv_SE"
)

```